FIELD NOTES & LESSONS LEARNED

Z80 DEV PIPELINE

Setting up a Z80 assembler development pipeline on Linux Mint with VSCodium, Pa ZesarUX emulator – and every obstacle e the way.

TARGET MACHINE: ZX Spectrum 48K
ASSEMBLER: Pasmo 0.5.3
EMULATOR: ZesarUX 12.1 Night Shift Editi
HOST OS: Linux Mint (Flatpak environmen
EDITOR: VSCodium (Flatpak)

Contents

01 Pipeline Overview — Tools and workflow

02 The Flatpak PATH Problem — Why VSCodium can't see system tools

03 ZesarUX Configuration — Machine IDs, ROM paths, and flags

04 The .zesaruxrc File — What it contains and what to watch out for

05 Errors Encountered & Solutions — A complete troubleshooting log

06 The Final Makefile — Annotated working configuration

07 Running from VSCodium — Tasks, shortcuts, and extensions

08 Reference & Quick-Look Tables — Machine IDs, ROM files, flags

CHAPTER 01

Pipeline Overview

The goal is a single `make run` command that assembles Z80 source code and launches it directly in the ZesarUX emulator. Here is how the pieces fit together.

THE TOOLCHAIN

The pipeline consists of four components working in sequence: the editor, the assembler, the build system, and the emulator.

COMPONENT	TOOL	ROLE
Editor	VSCodium (Flatpak)	Write Z80 assembly source with syntax highlighting via Z80 plugins
Assembler	Pasmo 0.5.3	Compile `.asm` source into a `.tap` tape image
Build system	GNU Make	Orchestrate assemble → run steps with a single command
Emulator	ZesarUX 12.1	Load and run the TAP file on a virtual ZX Spectrum

DIRECTORY LAYOUT

```
project/
├── src/
│   └── helloworld.asm    # Z80 assembly source
├── dist/
│   ├── helloworld.tap    # compiled tape image (generated)
│   └── helloworld.log    # assembler output log (generated)
└── Makefile
```

BUILD FLOW

Running `make run` triggers the following sequence:

```
make run
  └─► pasmo --name HelloWorld --tapbas src/helloworld.asm dist/helloworld.tap
  └─► zesarux --machine 48k --romfile ... --tape dist/helloworld.tap
```

NOTE

Pasmo is invoked with `--tapbas` which wraps the machine code in a BASIC loader — the standard way to distribute 48K Spectrum programs on tape.

CHAPTER 02

The Flatpak PATH Problem

The single biggest source of friction in this setup. VSCodium installed as a Flatpak runs inside a sandbox with a severely restricted PATH that cannot see most system-installed tools.

WHAT FLATPAK DOES TO PATH

When VSCodium is installed as a Flatpak, its integrated terminal does not inherit the normal system PATH. Instead it sees only a minimal subset:

```
# VSCodium Flatpak terminal PATH
/app/bin:/app/bin:/app/bin:/usr/bin:
/home/iian/.var/app/com.vscodium.codium/data/node/bin:
/home/iian/.var/app/com.vscodium.codium/data/cargo/bin:
/home/iian/.var/app/com.vscodium.codium/data/python/bin
```

```
# Normal system terminal PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:
/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

CRITICAL GAP

`/usr/local/bin` is absent from the Flatpak PATH. ZesarUX — compiled from source — installs to `/usr/local/bin/zesarux` and is therefore completely invisible to the Flatpak terminal.

THE FIX: FLATPAK-SPAWN --HOST

Flatpak provides a bridge command, `flatpak-spawn --host`, that escapes the sandbox and executes a command directly on the host system with the full system PATH. The solution is to prefix all external tool invocations in the Makefile with this command:

```
PASMO   ?= flatpak-spawn --host /usr/bin/pasmo
ZESARUX ?= flatpak-spawn --host /usr/local/bin/zesarux
```

Absolute paths are used alongside `flatpak-spawn --host` for reliability. Even with the host bridge active, relying on PATH resolution can be fragile — hardcoding the paths removes all ambiguity.

alternative approaches

ALTERNATIVE: NATIVE .DEB INSTALL OF VSCODIUM

The cleanest long-term solution is to install VSCodium natively rather than via Flatpak. The native install inherits the full system PATH and requires none of the `flatpak-spawn` workarounds. Native `.deb` packages are available from the VSCodium GitHub releases page at `github.com/VSCodium/vscodium/releases`.

```
# Install native VSCodium .deb
sudo dpkg -i codium_*.deb
```

Settings, extensions, and projects transfer seamlessly. This is the recommended path if the Flatpak friction becomes a recurring issue.

ALTERNATIVE: EXTEND PATH IN ~/.BASHRC

Adding the missing directories to `~/.bashrc` can help in some cases, though Flatpak sandboxing may still block access depending on its configuration:

```
export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:$PATH"
```

CHAPTER 03

ZesarUX Configuration

ZesarUX 12.1 has specific requirements for machine type, ROM files, and tape loading that differ from what one might expect or find documented in older guides.

MACHINE IDS

The `--machine` parameter requires exact ID strings. These are case-sensitive and do not follow an obvious naming pattern. The most relevant IDs for Spectrum development are:

ID	DESCRIPTION
`16k`	ZX Spectrum 16k
`48k`	ZX Spectrum 48k
`48kp`	ZX Spectrum+ 48k
`128k`	ZX Spectrum+ 128k
`P2`	ZX Spectrum +2
`P2A40`	ZX Spectrum +2A (ROM v4.0)
`P340`	ZX Spectrum +3 (ROM v4.0)

> **COMMON MISTAKE**
>
> `ZXSpectrum48`, `Spectrum48K`, and similar variations are all rejected. The correct value is simply `48k`.

ROM FILES

ZesarUX does not accept a `--rompath` directory parameter. Instead, use `--romfile` to specify a single ROM file directly. For the 48K Spectrum, only one ROM file is needed.

On a typical source-compiled ZesarUX installation, ROMs are found at:

```
/usr/local/share/zesarux/roms/48.rom       # 48K Spectrum
/usr/local/share/zesarux/roms/128-0.rom    # 128K Spectrum (bank 0)
/usr/local/share/zesarux/roms/128-1.rom    # 128K Spectrum (bank 1)
/usr/local/share/zesarux/roms/128p-0.rom   # Spectrum+ 128K (bank 0)
/usr/local/share/zesarux/roms/128p-1.rom   # Spectrum+ 128K (bank 1)
```

CASE SENSITIVITY

Linux filesystems are case-sensitive. ZesarUX looks for `48.rom` (lowercase). A file named `48.ROM` will not be found. Use `rename 'y/A-Z/a-z/' *.ROM` to convert if needed.

LOADING TAPE FILES

Despite what older documentation suggests, `--smartload` is not a valid command-line parameter in ZesarUX 12.1 — it is a GUI menu feature only. The correct flag for inserting a `.tap` file from the command line is `--tape`:

```
# WRONG — not a valid CLI parameter
--smartload dist/helloworld.tap

# CORRECT
--tape dist/helloworld.tap
```

ZesarUX's own help text confirms this: passing a filename without any flag causes ZesarUX to auto-detect the file type, equivalent to SmartLoad in the menu. However, using `--tape` explicitly is clearer and more reliable in scripts.

THE --EXPERTHELP FLAG

The standard `--help` output does not show all available parameters. Running `zesarux --experthelp` reveals additional options including `--romfile`. When troubleshooting, always check expert help before concluding that a feature is unavailable:

```
/usr/local/bin/zesarux --experthelp 2>&1 | grep -i rom
```

CHAPTER 04

The .zesaruxrc File

ZesarUX saves its GUI configuration to `~/.zesaruxrc` and reads it on startup. This file can accumulate stale or conflicting settings that interfere with command-line usage.

WHAT WAS FOUND IN THE FILE

Inspecting the file revealed several issues:

```
; Relevant excerpt from ~/.zesaruxrc
--smartloadpath "/usr/local/share/zesarux/roms/"  ← WRONG directory
--addlastfile "/usr/local/share/zesarux/roms/48.rom"
--addlastfile "/usr/local/share/zesarux/roms/48.rom"    ← duplicate
--addlastfile "/usr/local/share/zesarux/roms/48.rom"    ← duplicate
--addlastfile "/usr/local/share/zesarux/roms/128-0.rom"
--addlastfile "/usr/local/share/zesarux/roms/128p-1.rom"
```

ISSUES IDENTIFIED

PROBLEM 1 — MISPOINTED SMARTLOADPATH

`--smartloadpath` was set to the ROM directory. This setting tells ZesarUX where to look for tape files to smart-load, so pointing it at ROMs is semantically wrong and potentially confusing. It should point to your project's `dist/` folder.

PROBLEM 2 — DUPLICATE ADDLASTFILE ENTRIES

`48.rom` was listed three times under `--addlastfile`. This is harmless but indicates the file was accumulated across multiple GUI save operations. These entries track recently used files in the menu.

HOW SETTINGS INTERACT WITH CLI

Command-line parameters generally override the `.zesaruxrc` values for the same setting. However, some settings (particularly `--smartloadpath`) may persist from the config file even when not specified on the CLI. When unexpected behaviour occurs, the config file is a good place to investigate.

The config file can be bypassed entirely for a clean run using:

```
zesarux --noconfigfile --machine 48k --romfile ... --tape ...
```

> **TIP**
>
> `--noconfigfile` must be the *first* parameter — ZesarUX will ignore it if placed later in the argument list.

CHAPTER 05

Errors Encountered & Solutions

A complete log of every error hit during setup, in the order they were encountered, with root causes and solutions.

ERROR 1 — MAKE: ERROR 127 (PASMO NOT FOUND)

```
make: *** [Makefile:21: build] Error 127
```

Root cause: Exit code 127 means "command not found". Pasmo was not on the Flatpak terminal's restricted PATH, even though it was installed at `/usr/bin/pasmo` on the host system.

Solution: Use `flatpak-spawn --host /usr/bin/pasmo` in the Makefile.

next error

ERROR 2 — PORTAL CALL FAILED: NO SUCH FILE OR DIRECTORY (ZESARUX)

```
Portal call failed: Failed to start command: Failed to execute
child process "/usr/bin/zesarux" (No such file or directory)
```

Root cause: ZesarUX was compiled from source and installed to `/usr/local/bin/zesarux`, not `/usr/bin/zesarux`. Additionally, `/usr/local/bin` is absent from the Flatpak PATH entirely.

Solution: Use `flatpak-spawn --host /usr/local/bin/zesarux`.

next error

ERROR 3 — UNKNOWN MACHINE ZXSPECTRUM48

```
Error: Unknown machine ZXSpectrum48
```

Root cause: ZesarUX machine IDs do not follow a `ZXSpectrum48`-style convention. The correct ID is simply `48k`.

Solution: Replace `--machine ZXSpectrum48` with `--machine 48k`.

next error

ERROR 4 — UNKNOWN PARAMETER: --SMARTLOAD

```
Error: Unknown parameter : --smartload . Stopping parsing the rest of parameters
```

Root cause: `--smartload` and `--smartloadpath` are not valid ZesarUX CLI parameters in version 12.1. They are GUI-only features accessible from the File menu.

Solution: Replace `--smartload file.tap` with `--tape file.tap`. Remove `--smartloadpath` from the CLI invocation entirely.

next error

ERROR 5 — UNABLE TO OPEN ROM FILE 48.ROM

```
Error: Unable to open rom file 48.rom
```

Root cause: Without `--romfile` specified, ZesarUX looks for `48.rom` in its current working directory (and possibly a few standard paths). When launched from the project directory, it cannot find the ROM. There is no `--rompath` directory parameter in ZesarUX 12.1.

Solution: Use `--romfile /usr/local/share/zesarux/roms/48.rom` to provide the absolute path to the ROM file directly.

CHAPTER 06

The Final Makefile

The working, annotated Makefile incorporating all fixes discovered during the setup process.

MAKEFILE — FINAL WORKING VERSION

```
# ── tools ──────────────────────────────────────────────────────────
# flatpak-spawn --host escapes the VSCodium Flatpak sandbox so that
# host-system binaries can be invoked. Absolute paths are used for
# reliability — do not rely on PATH resolution inside the sandbox.
PASMO   ?= flatpak-spawn --host /usr/bin/pasmo
ZESARUX ?= flatpak-spawn --host /usr/local/bin/zesarux

# ── emulator ───────────────────────────────────────────────────────
# MACHINE: ZesarUX machine IDs are lowercase short codes.
#   48k | 128k | 16k | P2 | P340 etc.
# ROMFILE: --rompath is not supported; supply the ROM file directly.
MACHINE := 48k
ROMFILE := /usr/local/share/zesarux/roms/48.rom

# ── project ────────────────────────────────────────────────────────
NAME   := HelloWorld
SRC    := src/helloworld.asm
OUTDIR := dist
TAP    := $(OUTDIR)/helloworld.tap
LOG    := $(OUTDIR)/helloworld.log

.PHONY: all build run debug clean

all: build

$(OUTDIR):
        mkdir -p $(OUTDIR)

# Build TAP + capture assembler log into dist/helloworld.log
build: | $(OUTDIR)
        $(PASMO) --name $(NAME) --tapbas $(SRC) $(TAP) --log > $(LOG) 2>&1
        @echo "Built: $(TAP)"
        @echo "Log:   $(LOG)"

# Run in ZEsarUX
# --tape inserts the TAP into the virtual tape deck
```

```
# --machine and --romfile are both required
run: build
        $(ZESARUX) --machine $(MACHINE) --romfile $(ROMFILE) --tape $(TAP)

# Run with ZEsarUX debugger visible
debug: build
        $(ZESARUX) --machine $(MACHINE) --romfile $(ROMFILE) --tape $(TAP) --enabledebugg
er

clean:
        rm -rf $(OUTDIR)
```

SWITCHING TO 128K

To target the 128K Spectrum, only two lines need changing at the top of the Makefile:

```
MACHINE := 128k
ROMFILE := /usr/local/share/zesarux/roms/128-0.rom
```

ZesarUX will automatically load `128-1.rom` from the same directory as the second ROM bank.

CHAPTER 07

Running from VSCodium

Three ways to invoke make targets from inside the editor, ranging from simple to fully integrated.

OPTION 1: INTEGRATED TERMINAL

The simplest approach. Open the terminal with `Ctrl+` and type:

```
make run
make debug
make build
```

This is perfectly adequate for most workflows and requires no additional configuration.

OPTION 2: TASKS.JSON (RECOMMENDED)

VSCodium's task runner can wrap Makefile targets and bind them to keyboard shortcuts. Create `.vscode/tasks.json` in the project root:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build",
      "type": "shell",
      "command": "make build",
      "group": { "kind": "build", "isDefault": true },
      "presentation": { "reveal": "always", "panel": "shared" }
    },
    {
      "label": "Run in ZEsarUX",
      "type": "shell",
      "command": "make run",
      "group": "test",
      "presentation": { "reveal": "always", "panel": "shared" }
    },
```

```
    {
      "label": "Debug in ZEsarUX",
      "type": "shell",
      "command": "make debug",
      "group": "test",
      "presentation": { "reveal": "always", "panel": "shared" }
    }
  ]
}
```

SHORTCUT	ACTION
`Ctrl+Shift+B`	Run default build task (`make build`)
`Ctrl+Shift+P` → Tasks: Run Task	Pick any task from the list

OPTION 3: MAKEFILE TOOLS EXTENSION

Search the VSCodium extension marketplace for **Makefile Tools** (by Microsoft – compatible with VSCodium). It adds a dedicated sidebar panel listing all make targets, which can be clicked to run. No keyboard shortcuts needed.

> **FLATPAK NOTE**
>
> If using the Flatpak version of VSCodium and `tasks.json` is not picking up commands correctly, prefix the command with `flatpak-spawn --host` : set `"command": "flatpak-spawn --host make run"` .

CHAPTER 08

Reference & Quick-Look Tables

Consolidated reference material for the most commonly needed values and commands.

ZESARUX VALID CLI PARAMETERS (SUBSET)

PARAMETER	PURPOSE	EXAMPLE
`--machine id`	Select emulated machine	`--machine 48k`
`--romfile file`	Specify ROM file path	`--romfile /usr/.../48.rom`
`--tape file`	Insert TAP/TZX into tape deck	`--tape dist/game.tap`
`--snap file`	Load a snapshot file	`--snap game.z80`
`--enabledebugger`	Open debugger panel on start	–
`--noconfigfile`	Ignore ~/.zesaruxrc (must be first)	–
`--experthelp`	Show extended parameter list	–
`--version`	Print version and exit (must be first)	–

ROM FILES BY MACHINE

MACHINE ID	ROM FILE(S)
`48k`	`48.rom`
`128k`	`128-0.rom` , `128-1.rom`
`48kp`	`48.rom`
`P2`	`plus2-0.rom` , `plus2-1.rom`

MACHINE ID	ROM FILE(S)
P340	plus3-0.rom through plus3-3.rom

USEFUL DIAGNOSTIC COMMANDS

```
# Find where a binary actually lives
which pasmo
which zesarux

# Check what PATH the current shell sees
echo $PATH

# List all ROMs in the ZesarUX share directory
find /usr/local/share/zesarux -name "*.rom"

# Inspect the ZesarUX config file for ROM/path settings
grep -i rom ~/.zesaruxrc

# Test ZesarUX without loading the config file
zesarux --noconfigfile --machine 48k --romfile /usr/local/share/zesarux/roms/48.rom

# Check all expert parameters for ROM-related flags
/usr/local/bin/zesarux --experthelp 2>&1 | grep -i rom
```

END OF DOCUMENT · ZX SPECTRUM DEV PIPELINE · LINUX MINT